Projeto Interdisciplinar: Internet das coisas e robótica

Entrega 02

Grupo Enrolados

Guilherme Barioni
Iury Xavier da Silva Mangueira
Lilian Mercedes Paye Conde
Marcus Vinícius Duque
Murilo de Souza Vieira

Descrição da arquitetura final implementada

A segunda entrega apresenta a implementação final do Smart Ace, o sistema embarcado e IoT de apoio ao treinamento de tênis proposto na Entrega 01.

A solução foi implementada em quatro blocos integrados: a raquete instrumentada (MPU-6050 e ESP32), o aplicativo Android, o servidor de contas e sincronização (Node.js e MySQL) e a estação de visão computacional, que permanece como subsistema externo, conforme o escopo definido na primeira entrega. O aplicativo continua sendo o ponto de integração: recebe a classificação do golpe pela raquete, recebe os dados da bola pela estação de visão, aplica a lógica de gamificação, grava tudo localmente e sincroniza com o servidor.

Descrição dos blocos

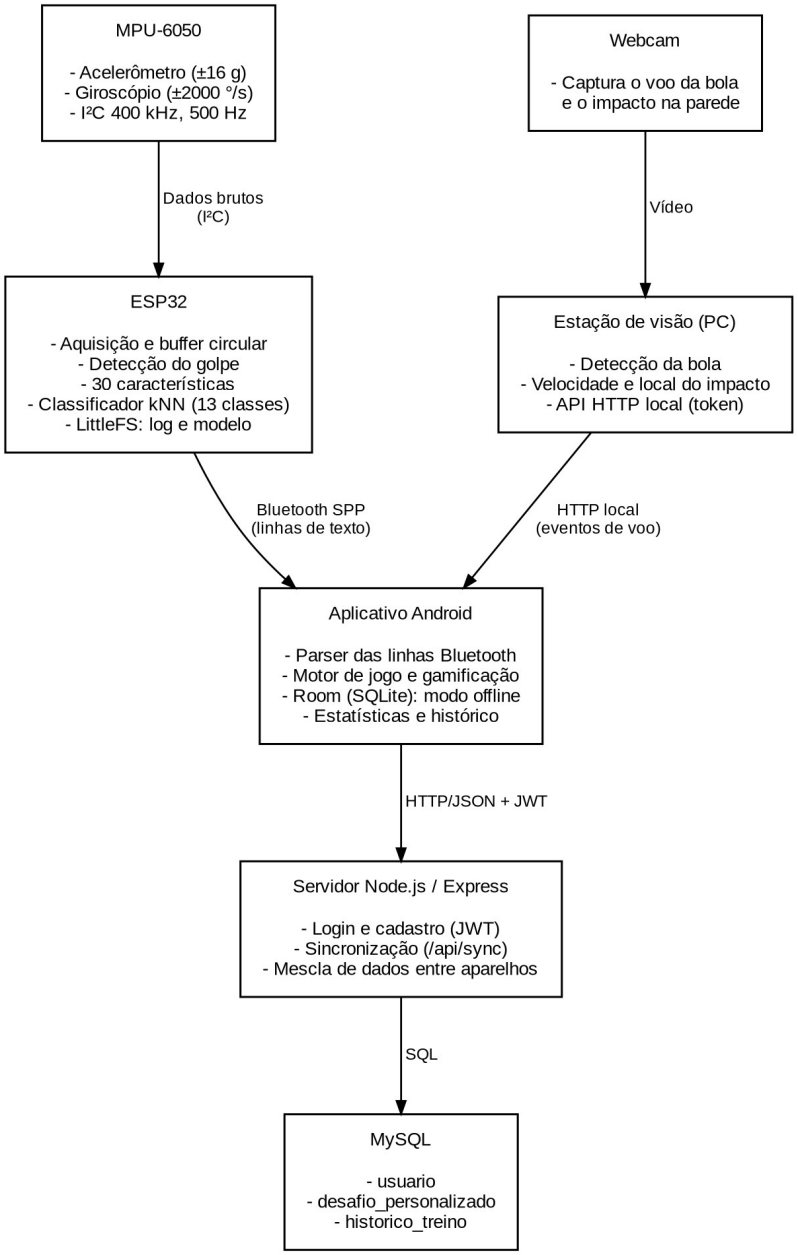
Raquete (MPU-6050 e ESP32). O MPU-6050 é lido por I²C a 400 kHz (SDA no GPIO21 e SCL no GPIO22), a 500 amostras por segundo, com escalas de ±16 g e ±2000 °/s. O ESP32 mantém um buffer circular com os 600 ms anteriores e, quando a aceleração passa de 4,5 g ou a velocidade angular de 350 °/s, completa uma janela de 1,1 s (550 amostras) com o backswing, o impacto e o follow-through. Dessa janela são extraídas 30 características (fusão acelerômetro-giroscópio pelo filtro de Mahony, fases do golpe, pico de choque da bola, eixo de rotação e trajetória) e o golpe é classificado por um kNN por classe, com pesos de Fisher e limite de rejeição individual. O modelo reconhece 13 classes: três tipos de saque, smash, forehand e backhand (drive, slice e voleio), amortida, lob e ruído/outro. O treino do modelo é feito no próprio dispositivo e o log de golpes fica gravado na memória flash (LittleFS).

Aplicativo Android. Desenvolvido em Java, conecta-se à raquete por Bluetooth Classic (perfil SPP), interpreta as linhas de texto enviadas pelo ESP32 e aciona o treino livre, o treino guiado e os jogos. Os dados ficam em um banco Room (SQLite), o que mantém o aplicativo totalmente funcional sem internet. Quando há conexão e sessão válida, o SyncManager envia desafios, histórico e progresso ao servidor.

Servidor. API REST em Node.js com Express e MySQL. Oferece cadastro e login com senha protegida por bcrypt e token JWT, além da rota de sincronização em lote (/api/sync), que grava os dados de forma idempotente e devolve o retrato atual da conta, permitindo ver o mesmo histórico em mais de um aparelho.

Estação de visão. Webcam e computador que detectam o voo da bola e o ponto de impacto na parede. O aplicativo consulta a estação por uma API HTTP local protegida por token, recebe apenas métricas e eventos e estima a diferença entre os relógios do computador e do celular para associar cada bola ao golpe correspondente.

Diagrama de Blocos do Sistema (versão final)



Ajustes em relação à proposta inicial

A Tabela abaixo resume o que mudou entre a Entrega 01 e a implementação final, junto com o motivo de cada ajuste.

Item	Entrega 01 (proposta)	Entrega 02 (implementado)	Motivo e impacto
Detecção do impacto	Placa piezoelétrica (limiar ≥ 2500) e botão de teste.	Somente MPU-6050: gatilho por aceleração/giro e pico de choque (2ª diferença da aceleração).	Menos hardware e fios na raquete; atende o baixo custo (RNF3).
Classificação	Três classes (saque, forehand, backhand) por critérios fixos.	13 classes com kNN, 30 características e rejeição de movimentos desconhecidos.	Cobre uma rotina de treino real, com golpes mais variados.

Item	Entrega 01 (proposta)	Entrega 02 (implementado)	Motivo e impacto
Câmeras	Velocidade, spin, trajetória e posição do impacto.	Webcam e estação no PC: velocidade da bola e impacto em grade na parede. Spin e trajetória completa não chegam ao app.	Dificuldade de implementação e custo elevado de computação

Máquina de Estados do Firmware

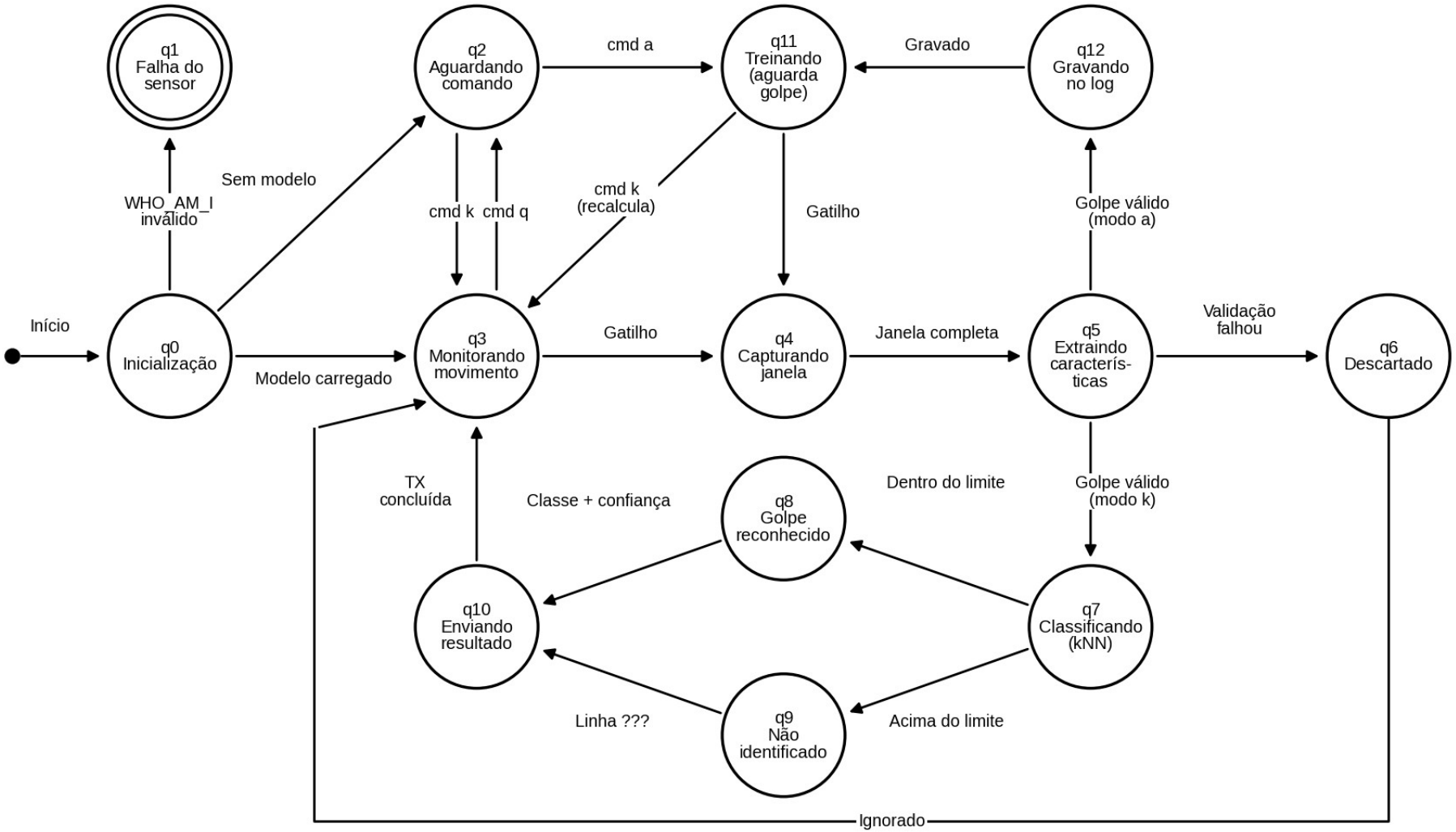


Figura 2 – Máquina de estados do detector de golpes (detector_golpes.ino, versão 2)

O diagrama acima substitui a máquina de estados da Entrega 01 e representa o comportamento real do firmware. A inicialização (q0) prepara o MPU-6050, a memória flash, a calibração e o Bluetooth. Se o sensor não responde, o sistema vai para q1 e para. Se já existe um modelo treinado, o dispositivo entra direto em q3, monitorando o movimento; caso contrário aguarda comandos em q2, e o aplicativo ou o terminal serial iniciam a classificação (k) ou o treino (a).

Em q3 cada amostra é comparada aos limiares de gatilho. Ao disparar, q4 completa a janela de 1,1 s e q5 valida o movimento e calcula as 30 características. Movimentos que não parecem golpe, como caminhada, raquete caindo ou batida curta, vão para q6 e são ignorados. Um golpe válido é classificado em q7: se a distância ao grupo mais próximo ficar dentro do limite da classe, o golpe é reconhecido (q8); se não, é marcado como não identificado (q9). Nos dois casos q10 envia a linha de resultado por Bluetooth (e pela porta USB) e o sistema volta a monitorar. No modo de treino, o golpe válido é gravado no log (q12) e o dispositivo aguarda o próximo (q11).

Identificação dos estados do sistema

q0 – Inicialização	Aloca os buffers, inicializa o MPU-6050, monta o LittleFS, carrega a calibração e o log de golpes e liga o Bluetooth RAQUETE-TENIS.
q1 – Falha do sensor	O registrador WHO_AM_I não retorna um valor esperado (0x68, 0x69, 0x70 ou 0x98). O sistema informa o erro e para.
q2 – Aguardando comando	Modo ocioso. O buffer circular continua sendo preenchido e o dispositivo aceita comandos pelo Bluetooth ou pela serial.
q3 – Monitorando movimento	Modo de classificação. A cada amostra (500 Hz) compara aceleração e velocidade angular com os limiares de gatilho.
q4 – Capturando janela	Reúne 300 amostras anteriores ao gatilho e 250 posteriores (1,1 s no total).
q5 – Extraindo características	Valida o movimento e calcula as 30 características do golpe.
q6 – Descartado	Sem rotação de golpe, pulso menor que 40 ms, movimento maior que 900 ms ou rotação total menor que 30°. Emite a linha “# ignorado”.
q7 – Classificando	Calcula a distância média aos 3 vizinhos mais próximos de cada classe ativa e compara com o limite de rejeição.
q8 – Golpe reconhecido	Classe de menor distância aceita; calcula a confiança e atualiza as estatísticas da sessão e da rotina de treino.
q9 – Não identificado	A distância ficou acima do limite da classe mais próxima. O golpe é informado como “não identificado”.
q10 – Enviando resultado	Escreve a linha de resultado no Bluetooth e na serial. Em seguida retorna a q3.
q11 – Treinando	Modo de treino com a classe escolhida (comando l). Aguarda um golpe para gravar.
q12 – Gravando no log	Grava as 30 características no arquivo /golpes.bin (LittleFS) e marca o modelo para ser recalculado.

Eventos (entradas)

Evento	**Símbolo**	**Descrição**
Início	**Inicio**	Reinício ou energização do ESP32.
Sensor inválido	**WHO_AM_I**	O MPU-6050 não responde com um identificador válido.
Modelo carregado	**Modelo**	O log tem pelo menos 2 classes com 5 ou mais golpes cada.

Evento	Símbolo	Descrição
Sem modelo	**SemModelo**	O log não atende ao mínimo de classes e amostras.
Comando de classificar	**cmd k**	O aplicativo ou o terminal envia “k”.
Comando de parar	**cmd q**	Envio de “q”: o dispositivo volta ao modo ocioso.
Comando de treinar	**cmd a**	Envio de “a”: inicia o modo de treino.
Gatilho	**Gatilho**	Aceleração ≥ 4,5 g ou velocidade angular ≥ 350 °/s, com pelo menos 800 ms desde o gatilho anterior (ajustável pelo comando t).
Janela completa	**Janela**	550 amostras coletadas.
Validação falhou	**Invalido**	O movimento não tem características de golpe (ver q6).
Golpe válido	**Golpe valido**	O movimento passou na validação (modo k segue para q7; modo a segue para q12).
Dentro do limite	**Dentro**	Distância dividida pelo limite da classe ≤ 1,3.
Acima do limite	**Acima**	Distância dividida pelo limite da classe > 1,3.
Linha enviada	**TX concluída**	O resultado foi escrito no Bluetooth e na serial.
Golpe gravado	**Gravado**	O golpe de treino foi salvo no log.

Código-fonte organizado e versionado

O código-fonte do Smart Ace está organizado em três módulos independentes, um por tecnologia, e é versionado com Git no repositório do grupo, acompanhado pelo GitHub Projects e pelas Issues.

Link do código fonte: https://github.com/2026-2-NCC6/Projeto4/tree/main/src/Entrega%202

A divisão em pacotes segue o requisito de código modular (RNF5): regras de jogo e protocolo da raquete (jogo), persistência local (database), comunicação com o servidor (backend) e utilidades compartilhadas (util). A tabela resume as responsabilidades.

Módulo	Arquivos principais	Responsabilidade
IoT	detector_golpes.ino	Aquisição do MPU-6050, detecção do golpe, extração de características, kNN, treino no dispositivo, persistência em LittleFS e protocolo de texto por Bluetooth.
Comunicação com a raquete	RaqueteBluetooth, ParserRaquete, ClasseGolpeRaquete, SwingEvento	Conexão RFCOMM/SPP, leitura de linhas, conversão do texto em eventos de golpe e catálogo das 13 classes.
Regras de jogo	MotorJogo, Recordes, Progressao, BolaEvento	Pareamento golpe × bola, pontos, combo, XP, níveis, recordes e modo Chefão.
Telas de treino	TreinoLivre, Treino, TreinoGuiadoLista, CriarDesafioActivity, JogoActivity	Treino livre, 12 missões offline, desafios personalizados, jogos e feedback visual.
Persistência local	AppDatabase, TreinoDAO, DesafioDAO, UsuarioDAO	Banco Room com migrações 1→6, histórico, desafios e usuários.
Sincronização	SyncManager, BackendApi, SessaoRemota, SenhaLocalUtil	Login/cadastro na API, token JWT, envio em lote e aplicação do retrato da conta; hash PBKDF2 da senha local.
Visão computacional	VisaoComputacionalClient	Status, sessão e eventos de voo da bola na estação do computador; ajuste de relógio.

Módulo	Arquivos principais	Responsabilidade
Backend	server.js, db.sql	Autenticação, validação de entrada, sincronização idempotente e regras de mescla.

Ao todo são 40 classes Java (7.541 linhas), 12 layouts XML, 1.162 linhas de firmware e 433 linhas de servidor com 7 rotas HTTP e 5 tabelas MySQL.

Evidência da integração entre sensores, processamento e atuadores

Ligações de hardware

Componente	Pino do ESP32	Função
MPU-6050 – VCC	3V3	Alimentação do sensor.
MPU-6050 – GND	GND	Referência comum.
MPU-6050 – SDA	GPIO21	Dados I²C (400 kHz, endereço 0x68).
MPU-6050 – SCL	GPIO22	Relógio I²C.
Antena Bluetooth	Interna (ESP32 clássico)	Perfil SPP com o nome RAQUETE-TENIS.

Cadeia de integração

A tabela mostra o caminho de um golpe, da leitura do sensor até o registro no servidor, com a função do código que realiza cada etapa.

Etapa	Entrada → saída	Processamento	Evidência no código
Sensor	Movimento → 14 bytes por amostra (aceleração, temperatura, giro).	Conversão de 16 bits, filtro digital interno de cerca de 184 Hz, 500 Hz.	mpuInit(), readRaw()
Aquisição	Amostras → janela de 550 amostras.	Buffer circular de 300 amostras, gatilho e período refratário de 800 ms.	pushRing(), ringToWindow(), loop()
Processamento	Janela → classe, confiança, km/h e g.	Validação, 30 características e kNN com rejeição.	computeFeatures(), classDist(), decide(), onSwing()
Transmissão	Resultado → linha de texto.	Escrita simultânea em Bluetooth SPP e USB.	out(), SerialBT
Recepção no app	Bytes RFCOMM → SwingEvento.	Leitura em thread própria, linhas de até 400 caracteres e expressão regular.	RaqueteBluetooth.ler(), ParserRaquete.parse()
Decisão e feedback	SwingEvento (e BolaEvento) → pontos, combo e tela.	Comparação com o golpe esperado, pareamento com a bola, XP e recordes.	MotorJogo, Treino, TreinoLivre
Persistência	Sessão → histórico local → servidor.	Gravação no Room e envio em lote com JWT.	TreinoDAO, SyncManager, /api/sync

Mensagens trocadas entre a raquete e o aplicativo

O firmware envia uma linha de texto por golpe. Os exemplos abaixo seguem exatamente o formato emitido pelo código:

```
#12   >> FH DRIVE      85%  ~92km/h a=12.3g
#13   ???  nao identificado (perto de FHD, d=3.20 lim=2.10) w=640
```

A primeira linha informa o número do golpe, a classe, a confiança, a velocidade estimada da cabeça da raquete e a aceleração de pico. A segunda indica que o kNN rejeitou outro movimento, por ficar acima do limite da classe mais próxima. Pelo caminho contrário, o aplicativo controla o firmware com comandos de uma letra:

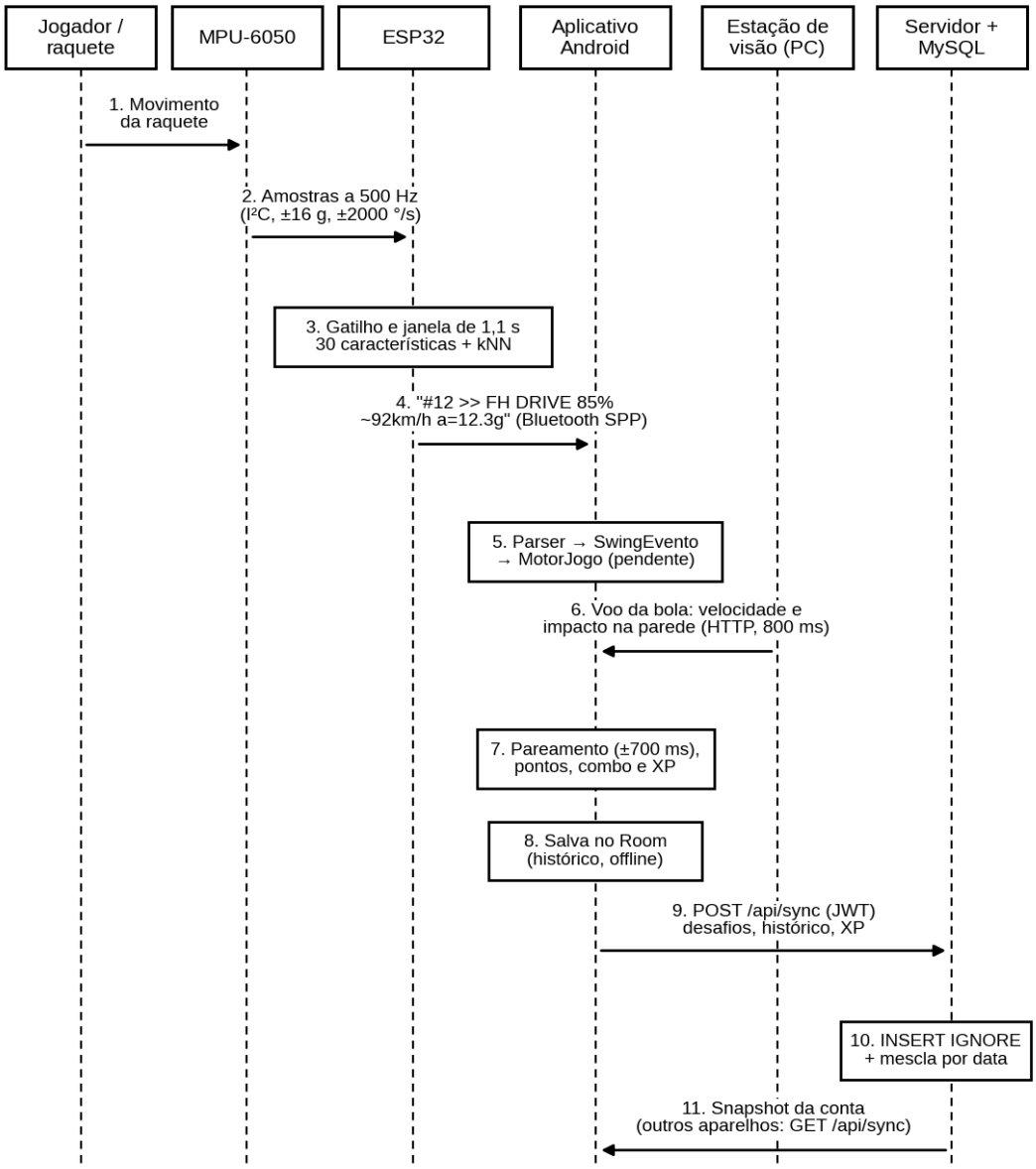
Comando	**Efeito no firmware (usado pelo app nas telas de treino)**
k	Entra no modo de classificação e responde “# CLASSIFICANDO”. Se não há modelo válido, responde “# sem modelo”.
q	Interrompe a classificação e responde “# parado”.
s	Devolve o resumo da sessão (golpes por classe, não identificados e ignorados).

Os demais comandos (a, l, e, m, g, o, x, t, u, i, w e outros) servem para treinar e ajustar o modelo pelo terminal serial ou Bluetooth e estão descritos na ajuda do firmware (comando h).

Atuação do sistema

Na implementação final, a atuação acontece em duas direções. Do sistema para o jogador, o aplicativo exibe a contagem regressiva de início (3, 2, 1 e “VAI!”), o nome do último golpe, a confiança, a aceleração, a velocidade estimada, a pontuação, o combo e o alvo da parede. Do aplicativo para a raquete, os comandos k, q e s ligam, desligam e consultam o classificador.

Demonstração do fluxo completo de dados e ações do sistema



O quadro abaixo descreve, passo a passo, uma sessão de Treino Guiado (por exemplo, a missão Rei do Saque), do login ao histórico visível em um segundo aparelho.

Nº	Ação	O que acontece
1	Login	O app valida a senha localmente (PBKDF2, 120.000 iterações) e, havendo internet, autentica na API (POST /api/auth/login) e guarda o token JWT.
2	Escolha do treino	A lista reúne 12 missões offline e os desafios personalizados do usuário, lidos do Room e sincronizados com o servidor.
3	Conexão com a raquete	O app procura o dispositivo pareado RAQUETE-TENIS e abre o canal RFCOMM SPP.
4	Início	Após a contagem 3-2-1, o app envia “k”. O ESP32 entra em q3 e confirma com “# CLASSIFICANDO”.
5	Golpe	A raquete captura a janela de 1,1 s, extrai as características, classifica e envia a linha de resultado (aproximadamente 0,8 s após o golpe).

Nº	Ação	O que acontece
6	Avaliação	O app compara a classe recebida com o golpe esperado da sequência. Cada acerto vale 100 pontos e 3 acertos seguidos geram bônus de combo. Golpes rejeitados não avançam a sequência.
7	Câmera (opcional, nos jogos)	A estação informa o voo da bola. O app pareia golpe e bola por tempo (janela de 700 ms), valida o impacto na parede e usa a velocidade da bola e a zona atingida para pontuar.
8	Encerramento	O app envia “q”, calcula pontos e XP e grava a sessão no Room com um identificador único (syncId).
9	Sincronização	O SyncManager envia desafios, histórico e progresso em um único POST /api/sync. O servidor valida o token, grava sem duplicar e devolve o retrato da conta.
10	Outro aparelho	Ao entrar com a mesma conta, o segundo aparelho baixa o retrato da conta e passa a exibir o mesmo histórico e as mesmas estatísticas.

Se o servidor ou a estação de visão estiverem indisponíveis, o treino não é interrompido. A falha de sincronização é registrada e os dados permanecem no celular até a próxima conexão, e os jogos podem ser executados sem câmera.

Registro dos testes realizados e validação dos requisitos

Testes automatizados da lógica do aplicativo

Em 28/10/2026 foram executados 23 testes sobre os arquivos-fonte reais do pacote jogo (ParserRaquete, ClasseGolpeRaquete, SwingEvento, BolaEvento, MotorJogo, Recordes e Progressao). As linhas de entrada seguem o formato exato emitido pelo firmware. O programa de teste (Testes.java) acompanha esta entrega. Resultado: 23 de 23 testes aprovados.

ID	Teste	Esperado	Obtido	Resultado
T01	Parser reconhece as 13 classes do firmware (código, confiança, km/h e g)	13/13	13/13	**OK**
T02	Família de “FH DRIVE”	FOREHAND	FOREHAND	**OK**
T03	Instante do contato = chegada − 800 ms	800	800	**OK**
T04	Golpe rejeitado pelo kNN vira “não identificado”	true	true	**OK**
T05	Linhas de status e diagnóstico não geram golpe	7/7	7/7	**OK**
T06	Classe RUIDO/OUTRO não pontua	false	false	**OK**
T07	corresponde(SAQUE, SKK): família genérica aceita qualquer saque	true	true	**OK**
T08	corresponde(FHD, FHS): classe exata diferente não conta	false	false	**OK**
T09	corresponde(BACKHAND, BHV): voleio pertence à família	true	true	**OK**
T10	Nome antigo “Forehand drive” é normalizado para FHD	FHD	FHD	**OK**
T11	Nível com 0 XP	1	1	**OK**
T12	Nível com 150 XP	2	2	**OK**
T13	Nível com 600 XP	3	3	**OK**
T14	Multiplicador de combo com 0, 3, 9 e 30 acertos	1,0 | 1,5 | 2,5 | 4,0	1,0 | 1,5 | 2,5 | 4,0	**OK**

ID	Teste	Esperado	Obtido	Resultado
T15	Sem câmera: confiança ≥ 60% é validado (100 pontos)	VALIDADO/100	VALIDADO/100	**OK**
T16	Sem câmera: confiança < 60% é sem leitura e zera o combo	SEM_LEITURA/0	SEM_LEITURA/0	**OK**
T17	Combo de 3 acertos eleva o multiplicador para 1,5	1,5	1,5	**OK**
T18	Com câmera: golpe fica pendente aguardando a bola	1	1	**OK**
T19	Bola na parede dentro da janela valida o golpe	VALIDADO	VALIDADO	**OK**
T20	Velocidade da bola e zona do impacto são repassadas	70 km/h / zona 2	70 km/h / zona 2	**OK**
T21	Bola vista que não chegou à parede	BOLA_SEM_PAREDE	BOLA_SEM_PAREDE	**OK**
T22	Sem leitura da bola até o timeout de 3,5 s	SEM_LEITURA	SEM_LEITURA	**OK**
T23	Bola fora da janela de 700 ms não é pareada	1 pendente	1 pendente	**OK**

Verificações por análise do código

Os itens abaixo foram verificados na leitura do código-fonte.

Comportamento	Evidência no código
Sincronização sem duplicar sessões	Chave única (usuario_id, sync_id) em historico_treino e INSERT IGNORE no servidor.
Isolamento entre usuários	O identificador do usuário vem sempre do token JWT validado, nunca do corpo da requisição.
Proteção de senhas	bcrypt com custo 12 no servidor e PBKDF2 (120.000 iterações, salt de 16 bytes) no aplicativo.
Limite de tentativas de acesso	30 requisições de login/cadastro por IP a cada 15 minutos (express-rate-limit).
Conflito de edição entre aparelhos	Desafios mesclados pelo carimbo atualizado_em; vence a versão mais recente.
Treino não depende da internet	Falha de sincronização é capturada no SyncManager e não interrompe a tela de treino.
Tempo de resposta da API	Tempos limite de 1,0 s (conexão) e 1,8 s (leitura) no cliente HTTP do aplicativo.
Compatibilidade com o firmware anterior	ParserRaquete aceita as linhas antigas (SAQUE, FOREHAND, BACKHAND e w=).

Testes de bancada e de campo

Os testes abaixo dependem do hardware e de execuções reais de golpes e devem ser registrados pelo grupo na coluna de resultado. Os critérios de aceitação são propostos e podem ser ajustados pela equipe.

ID	Teste	Procedimento	Critério proposto	Resultado
B1	Precisão por classe	Gravar pelo menos 20 golpes por classe (comando a) e consultar o relatório do modelo (comando m).	Precisão LOO ≥ 85% e nenhuma classe abaixo de 70%.	**OK**

ID	Teste	Procedimento	Critério proposto	Resultado
B2	Rejeição de não-golpes	Caminhar, quicar a raquete e bater a raquete na mão com o classificador ligado.	No máximo 1 falso golpe a cada 20 eventos.	**OK**
B3	Latência golpe → tela	Filmar o golpe e a tela do app e medir o intervalo.	≤ 1,5 s (RNF1).	**OK**
B4	Alcance e reconexão Bluetooth	Afastar o celular 5 e 10 m; desligar e religar o Bluetooth.	Reconexão pelo botão do app sem reiniciar o ESP32.	**OK**
B5	Dois aparelhos (RF11)	Concluir uma sessão no aparelho A e abrir a mesma conta no aparelho B.	Sessão visível no B, sem duplicação.	**OK**
B6	Golpe × bola (câmera)	Executar 30 golpes contra a parede com a estação ativa.	≥ 90% das bolas pareadas ao golpe correto.	**OK**

Validação dos requisitos

ID	Requisito (resumo)	Situação	Evidência e observação
RF01	Categorizar o golpe com o MPU-6050.	**Atendido**	13 classes por kNN com rejeição; T01 a T06. Precisão em campo a registrar (B1).
RF02	Enviar dados ao app via Bluetooth.	**Atendido**	Bluetooth SPP com linhas de texto; ParserRaquete e T01.
RF03	Velocidade, rotação e local do impacto pelas câmeras.	**Parcial**	Velocidade da bola e impacto na parede chegam ao app; rotação (spin) não foi integrada.
RF04	Envio de dados por Bluetooth ao app.	**Atendido**	Mesma evidência do RF02.
RF05	Login e cadastro.	**Atendido**	Login local e na API (/api/auth/login e /api/auth/register).
RF06	Criar e visualizar sessões de treino.	**Atendido**	Desafios personalizados, 12 missões offline, Treino Livre e Treino Guiado.
RF07	Sessões interativas com gamificação.	**Atendido**	Pontos, combo, XP, níveis, recordes e Chefão; T11 a T17.
RF08	Visualizar e analisar os dados.	**Atendido**	Tela de estatísticas calculada a partir do Room.
RF09	Funcionar offline.	**Atendido**	Room como base principal; sincronização opcional.
RF10	Enviar perfil e treinos a servidor externo.	**Atendido**	POST /api/sync. Nos testes o servidor roda na rede local do grupo.
RF11	Ver os dados em vários aparelhos.	**Atendido**	Retrato da conta em GET/POST /api/sync. Validação com 2 aparelhos pendente (B5).
RNF1	Baixo tempo de envio e resposta.	**Parcial**	O resultado chega cerca de 0,8 s após o golpe, por causa da janela de captura. Medição em B3.
RNF2	Envio seguro e criptografado.	**Parcial**	Senhas com hash, JWT e Helmet; porém a API é usada por HTTP e o canal SPP não tem criptografia própria.
RNF3	Baixo custo.	**Atendido**	ESP32, MPU-6050 e webcam; o piezo e o LED foram eliminados.
RNF4	Interface intuitiva.	**Atendido**	Fluxos guiados e feedback visual. Sem teste formal de usabilidade.
RNF5	Código modular.	**Atendido**	Pacotes por responsabilidade no app; firmware ainda em arquivo único.

Backlog final atualizado

O backlog da Entrega 01 foi revisado item a item e acrescido das tarefas criadas durante o desenvolvimento. A coluna de situação indica o que foi concluído, o que foi entregue apenas em parte e o que permanece pendente.

Itens do backlog inicial

ID	Nome	Prioridade	Situação	Observação
01	Configuração do ESP-32	**Alta**	**Concluído**	Ambiente Arduino com esquema de partição Huge APP.
02	Integração do MPU6050	**Alta**	**Concluído**	I²C a 400 kHz, 500 Hz, ±16 g e ±2000 °/s.
03	Leitura dos sensores MPU	**Alta**	**Concluído**	Seis eixos e temperatura em uma leitura de 14 bytes.
04	Calibração dos sensores	**Alta**	**Parcial**	O firmware lê e aplica a calibração gravada; a rotina que a grava não consta no sketch final.
05	Sensor de impacto	**Alta**	**Concluído**	Alterado: impacto detectado pelo próprio MPU-6050 (pico de choque); piezo descartado.
06	Detecção de movimento	**Alta**	**Concluído**	Gatilho por aceleração/giro e validação por duração e rotação.
07	Filtragem dos dados	Médio	**Concluído**	Filtro digital do sensor, média móvel e filtro de Mahony.
08	Extração de características	**Alta**	**Concluído**	30 características por golpe.
09	Classificação dos golpes	**Alta**	**Concluído**	13 classes, kNN com pesos de Fisher e rejeição.
10	Testes de classificação	**Alta**	**Parcial**	Validação leave-one-out embutida; precisão em campo a registrar (B1, B2).
11	Comunicação ESP32-Aplicativo	**Alta**	**Concluído**	Bluetooth SPP e parser no app.
12	Captura das câmeras	**Alta**	**Parcial**	Webcam única na estação do computador.
13	Processamento de imagens	**Alta**	**Parcial**	Executado na estação; o desempenho em tempo real depende do hardware do PC.
14	Detecção da bola	**Alta**	**Parcial**	Detecta o voo da bola; falhas de leitura são toleradas pelo jogo.
15	Cálculo da velocidade e spin	**Alta**	**Parcial**	Velocidade máxima e média entregues; spin não integrado.
16	Detecção do impacto	**Alta**	**Concluído**	Impacto na parede em grade (coluna e linha) informado ao app.
17	Integração dos dados	**Alta**	**Concluído**	Pareamento golpe × bola por tempo e ajuste de relógio.
18	Desenvolvimento do aplicativo	**Alta**	**Concluído**	40 classes Java e 12 telas.
19	Tela de resultados	Médio	**Concluído**	Resumo da sessão, pontuação, XP e recordes.
20	Sistema de gamificação	**Alta**	**Concluído**	Pontos, combo, XP, níveis e Chefão.
21	Histórico de sessões	Médio	**Concluído**	Histórico no Room e na tela de estatísticas.
22	Servidor	**Alta**	**Concluído**	Node.js/Express com JWT.
23	Persistência dos dados	**Alta**	**Concluído**	MySQL com 5 tabelas.
24	Integração completa	**Alta**	**Parcial**	Raquete, app e servidor integrados; a câmera depende da estação no PC.
25	Testes de integração	Médio	**Parcial**	Testes da lógica de integração (T01 a T23); falta teste ponta a ponta.

ID	Nome	Prioridade	Situação	Observação
26	Testes de campo	Médio	**Pendente**	Plano B1 a B6 definido; execução e registro pendentes.
27	Documentação técnica	Médio	**Concluído**	Entregas 01 e 02.

Itens incluídos durante o desenvolvimento

ID	Nome	Prioridade	Situação	Observação
28	Classificador kNN de 13 classes com rejeição	**Alta**	**Concluído**	Substitui a classificação por regras fixas.
29	Treino do modelo no dispositivo (LittleFS)	**Alta**	**Concluído**	Até 400 golpes em /golpes.bin.
30	Login e cadastro com JWT	**Alta**	**Concluído**	Contas locais e remotas.
31	Modo offline e sincronização em lote	**Alta**	**Concluído**	Room como fonte principal.
32	Treino Livre, Treino Guiado e desafios personalizados	**Alta**	**Concluído**	12 missões offline e criação de desafios.
33	Jogos Rali de Combo e Chefão	**Média**	**Concluído**	Usam raquete e câmera, ou só raquete.
34	Transporte seguro (HTTPS/TLS)	**Alta**	**Pendente**	App e servidor usam HTTP na rede local.
35	Salvar métricas por classe no servidor	**Média**	**Pendente**	Falta a coluna metricasGolpesJson em historico_treino.
36	Spin da bola integrado ao aplicativo	**Média**	**Pendente**	Requer evolução da estação de visão.
37	Atuador físico na raquete (LED ou vibração)	**Baixa**	**Pendente**	Recupera o feedback previsto na Entrega 01.
38	Migração para BLE com protocolo estruturado	**Média**	**Pendente**	Substitui o texto com expressão regular.
39	Testes automatizados do servidor e pacote reproduzível	**Média**	**Pendente**	package.json, .env.example e README.

Resumo: dos 39 itens do backlog, 24 estão concluídos, 8 parcialmente entregues e 7 pendentes. Entre os 27 itens originais, 18 foram concluídos, 8 ficaram parciais e 1 segue pendente.

As pendências de maior prioridade para a próxima etapa são: transporte seguro entre o aplicativo e o servidor (item 34), registro dos testes de campo e da precisão por classe (itens 10 e 26), integração do spin da bola (item 36) e recuperação do feedback físico na raquete (item 37).

Análise crítica das limitações e propostas de melhoria

A análise abaixo separa as limitações por parte do sistema. Cada uma foi identificada no código ou nos testes e vem acompanhada de uma proposta de melhoria.

Raquete e classificação

Limitação	Consequência	Proposta de melhoria
O firmware classifica depois de completar a janela de 1,1 s (500 ms após o gatilho).	O resultado chega cerca de 0,8 s depois do golpe; o feedback não é instantâneo (RNF1).	Classificar de forma incremental, logo após o pico de impacto, com janela deslizante.

Limitação	Consequência	Proposta de melhoria
O modelo aprende com a montagem de cada raquete e jogador (mínimo de 5 golpes por classe, log de até 400 golpes).	A precisão pode cair se o sensor mudar de posição ou de jogador; a generalização não foi validada.	Montar um conjunto de dados com vários jogadores, normalizar a orientação do sensor e validar por jogador.
A velocidade da raquete é estimada por ω × 0,60 m (braço fixo).	É uma estimativa grosseira e não é a velocidade da bola.	Medir o braço de cada jogador (comando w) e comparar com a velocidade da câmera.
O firmware é um único arquivo e usa um só laço para amostrar, classificar e transmitir.	Durante a extração das características não há amostragem; manutenção mais difícil (RNF5).	Separar amostragem e processamento em tarefas do FreeRTOS e modularizar em arquivos .h e .cpp.

Comunicação e visão computacional

Limitação	Consequência	Proposta de melhoria
O protocolo é texto livre lido com expressão regular, sobre Bluetooth Classic (SPP).	Qualquer mudança no formato quebra o app; o SPP não funciona em iPhone.	Migrar para BLE com mensagens estruturadas, versionadas e com verificação de erro.
O instante do contato é estimado como a chegada da linha menos 800 ms.	O pareamento com a bola pode falhar com bolas muito próximas.	Enviar o carimbo de tempo do ESP32 e sincronizar os relógios.
A estação de visão usa uma webcam, exige parede configurada e depende do hardware do computador. O modelo de tracking ainda roda abaixo do ideal na máquina do grupo.	Sem spin nem trajetória completa; leituras perdidas em ambientes mal iluminados.	Otimizar o modelo (quantização, GPU, menor resolução), avaliar uma segunda câmera e integrar o spin.

Aplicativo, servidor e segurança

Limitação	Consequência	Proposta de melhoria
A API do app e a da estação usam HTTP; o endereço do servidor é fixo no código e o CORS aceita qualquer origem por padrão.	RNF2 só em parte: dados e token trafegam sem TLS.	Publicar a API com HTTPS (proxy reverso e certificado), restringir o CORS e criar tela de configuração do servidor.
O token JWT fica em SharedPreferences sem criptografia, e senhas antigas em texto puro ainda são aceitas no login local.	Risco em aparelho comprometido.	Usar EncryptedSharedPreferences/Keystore e remover a compatibilidade com senhas em texto puro.
O servidor não guarda as métricas por classe (metricasGolpesJson) e devolve a conta inteira a cada sincronização (limite de 1.000 itens por lista).	O app precisa duplicar esses dados em outro campo; custo cresce com o histórico.	Criar a coluna no esquema e sincronizar de forma incremental pelo carimbo atualizado_em.
As telas concentram regras e interface (Treino.java tem 1.455 linhas) e só o pacote jogo tem testes automatizados; o servidor não tem testes.	Mais difícil de manter e de testar.	Adotar MVVM, ampliar os testes (Room, API com Jest/supertest) e executá-los em integração contínua.
O pacote do servidor não inclui package.json nem .env.example, e uma mensagem de erro cita sql/schema.sql, mas o arquivo se chama db.sql.	Dificulta reproduzir o ambiente.	Incluir os arquivos de configuração e um README com os passos de instalação.

Validação

Limitação	Consequência	Proposta de melhoria
Os testes de campo, a precisão por classe e a usabilidade ainda não foram medidos com vários jogadores.	Os requisitos RF01 e RNF1 estão atendidos pelo desenho, mas sem números de campo.	Executar B1 a B6, registrar a matriz de confusão e aplicar um teste de usabilidade (por exemplo, SUS).

Considerações finais

A implementação final do Smart Ace integra, em um fluxo contínuo, a raquete instrumentada, o aplicativo, o servidor e a estação de visão. O ESP32 identifica o golpe entre 13 classes a partir dos dados do MPU-6050, o aplicativo transforma essa informação em feedback, pontos e histórico, e o servidor mantém a conta sincronizada entre aparelhos. A lógica de integração do aplicativo foi validada por 23 testes automatizados, e os requisitos funcionais foram atendidos, com exceção da rotação da bola, entregue apenas em parte.

Em relação à proposta inicial, o projeto simplificou o hardware (sem piezo), ampliou o reconhecimento de golpes e deslocou o feedback ao jogador para o aplicativo. Ficam como principais pendências o transporte seguro dos dados, o registro dos testes de campo e a evolução do subsistema de visão. Essas tarefas já constam no backlog final e orientam a continuidade do projeto.